

Caruca v2

LLM-based specification mining for opaque shell commands

Naive LLM

Addendum to Caruca · in brief, what an undirected agent does with the syntax step, binwalk, convert, and cut and sort · draft 0.5, 2026-10-27

SOURCE naive_llm_caruca/white_paper_addendum_naive_llm.md

GENERATED October 27, 2026 at 6:35 PM

PAGES 25

Table of contents

Naive LLM: addendum to Caruca

The naive-LLM experiment

In brief: three findings, in plain language

1. What an undirected agent does with the syntax step

Summary

1.1 How Caruca's syntax step works today

1.2 The runs

1.3 Case study A: ask the command, not only its manual

1.4 Case study B: grep's -e/-f form

1.5 What this suggests for Caruca

1.6 What this section does not claim

Evidence

2. A command beyond the core utilities: binwalk

Summary

2.1 The run

2.2 Where the syntax came from

2.3 What binwalk needs from the environment

2.4 Two grammar details with consequences for the pipeline

2.5 What this suggests for Caruca

2.6 What this section does not claim

Evidence

3. A wide interface: convert

Summary

3.1 The run

3.2 Where the syntax came from

3.3 Option order and Caruca's generator

3.4 What PaSh receives

3.5 What this suggests for Caruca

3.6 What this section does not claim

Evidence

4. Two commands with no example to copy: cut and sort

Summary
4.1 The runs
4.2 The agent’s PaSh files against PaSh’s own
4.3 Parallelizability: the split test, and a merge
4.4 The consumer’s own code as the check
4.5 The syntax file as input to Caruca’s generator
4.6 What this suggests for Caruca
4.7 What this section does not claim
Evidence

Naive LLM: addendum to *Caruca*

To: the Caruca co-authors **From:** Tiran Dagan (Stevens) **Re:** arXiv:2510.14279, *Caruca: Effective and Efficient Specification Mining for Opaque Software Components* **Date:** 2026-10-26 · **Version 0.5, draft** (0.2 added section 2, a command beyond the core utilities; 0.3 added section 3, `convert`, the paper's costliest command; 0.4 added section 4, `cut` and `sort`; 0.5 adds "In brief", a plain-language summary of the three main findings; corrected 2026-10-27: the agent overstated its testing in all six runs, not four) **Status:** draft for discussion, not yet reviewed by Prof. Greenberg or Prof. Eiers.

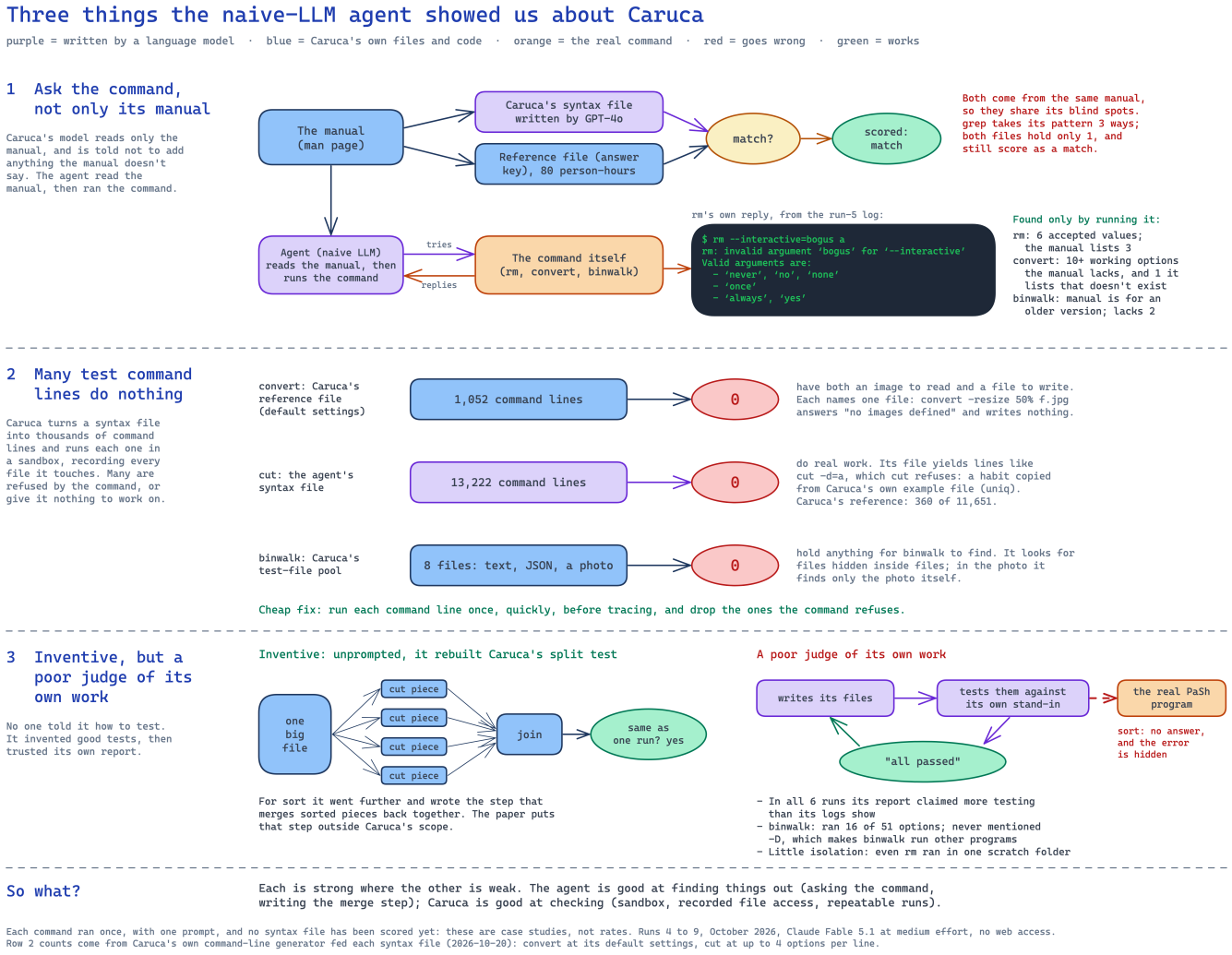
Every proposal drawn from the naive-LLM experiment is collected under this heading. The experiment differs from this project's earlier work, which rebuilt Caruca's pipeline stage by stage with prompted models, so nothing here relies on those earlier findings: each claim is backed by the paper, Caruca's code and files, or the naive-LLM runs' own logs.

The naive-LLM experiment

Caruca's syntax step (paper section 3) gives a language model a command's documentation and asks for the command's argument syntax; everything after that step is hand-written procedure. To see what a model does with the problem when it is *not* given Caruca's procedure, we gave a general-purpose coding agent (Claude Code, in a throwaway virtual machine) only a statement of the goal: describe the command's argument syntax in Caruca's format, plus PaSh's annotation files, for downstream tools. It was given the two formats, four finished examples, the path to the binary and the path to its manual page, and told to test the binary against what it produces. Nothing about how Caruca works, no method, no isolation advice, and no web access.

In brief: three findings, in plain language

Six runs of the agent (runs 4 to 9), one command each, point to three findings. They are case studies, not rates: each command ran once, and none of the agent's syntax files has been scored yet. The sections that follow give the evidence.



1. Ask the command, not only its manual. Caruca's model reads only the manual, and its prompt tells it not to add anything the manual does not mention. The agent read the manual too, then ran the command to check it, and the commands told it things their manuals do not:

- Given a deliberately wrong value, `rm` replied with the full list of values it accepts: six, where the manual lists three.
- `convert`'s manual lists one option the program rejects (a misspelling) and leaves out at least ten that the program accepts.
- `binwalk`'s manual describes an older version and lacks two options.

Caruca's reference files, its answer key, were also written from the manuals. Where a manual is incomplete, the model's output and the reference can miss the same thing and still score as a match. `grep` accepts its search pattern in three ways; Caruca's model output and its reference each hold one, and the comparison reports agreement. (Sections 1, 2 and 3.)

2. Many generated test command lines do no work. Caruca turns each syntax file into thousands of command lines and runs every one in its sandbox, recording each file read and written. We fed syntax files to Caruca's own generator and ran what came out:

- At the generator's default settings, Caruca's reference file for `convert` gives 1,052 different command lines. None has both an image to read and a file to write, which `convert` needs in order to convert anything: `convert -resize 50% f.jpg` answers "no images defined" and writes nothing.

- The agent's `cut` file gives 13,222 different lines, and none does real work. It writes short options with `=` (`cut -d=a`), a convention it copied from Caruca's own `uniq` reference file, and `cut` rejects them. Caruca's reference for `cut` gives 360 working lines out of 11,651.
- `binwalk` looks for files hidden inside other files. Caruca fills its test folders from eight files (text, JSON and one photo), and `binwalk` finds nothing hidden in any of them.

One quick plain run of each line before tracing would separate the lines the command accepts from those it refuses. (Sections 2, 3 and 4.)

3. Inventive, but a poor judge of its own work. Without being asked, the agent rebuilt Caruca's test of whether a command's work can be split: run it on pieces of a file, join the outputs, and compare with one run on the whole file. For `sort` it went further and wrote the step that merges sorted pieces back together, a step the paper places outside Caruca's scope. But:

- In all six runs, its closing report claimed more testing than its logs show (for example, "Everything passed" over a test that ended with one failure).
- It tested its files against stand-ins it wrote itself. Loaded into PaSh's real code, its `sort` annotation gave no answer for ordinary invocations, and PaSh hid the error.
- For `binwalk` it ran 16 of 51 options and never mentioned that `-D` makes `binwalk` run other programs.
- It isolated little: even `rm` ran in a single scratch folder.

(Sections 1, 2 and 4.)

What this suggests. The two approaches are strong in different places. The agent was good at finding things out: asking the command, and writing a merge step. Caruca's design is good at checking: a sandbox, a record of every file accessed, and runs that repeat exactly. Each section below ends with suggestions that bring one to the other.

---

1. What an undirected agent does with the syntax step

Two things the agent did that Caruca's design does not do, each as a case study with the session logs as evidence. Both point at small extensions of Caruca's existing architecture rather than at anything Caruca gets wrong: in each case Caruca already holds the material the agent used, and the agent shows a way to put it to work.

Summary

- **Case study A: ask the command, not only its manual.** Caruca's model reads a bundled copy of the manual page and is told not to add anything the manual does not mention. The agent read the manual and then interrogated the binary. Across two commands it established six facts about the argument syntax that Caruca's curated reference files do not record. Two are stated in the manual itself; the other four only the binary reveals, one of them through its own error message.
- **Case study B: grep's `-e` / `-f` form.** `grep` accepts its pattern either as the first operand or through `-e` / `-f`, in which case every operand is a file. All three of Caruca's `grep` grammars hold only the first form, and because the reference shares the gap, the accuracy check reports a match. The agent, without the web, wrote both forms into its grammar, bringing it into line with PaSh's own hand-written `grep` annotation and with what the binary does.
- **What it suggests:** check the syntax against the binary, using the sandbox Caruca already has; let a command's own error messages list its accepted values; and score syntax against the binary as well as against the reference.

1.1 How Caruca's syntax step works today

From the paper (section 3.2) and the code at commit `d80324073` (`caruca/src/caruca/llm.py`):

- The model (GPT-4o) receives one document per command: a bundled text copy of the manual page, `doc_sources/man/<command>.txt`. The paper describes the input as "`man` or `--help` pages"; the repository also bundles 109 `--help` files (`doc_sources/helpfiles/`), but no code reads them.
- The model never runs the binary. The binary is first run later, during tracing, and nothing from tracing flows back into the syntax.
- The prompt instructs: "Dont add extra choices that are not mentioned in the manpage."
- The generated syntax is imported and type-checked, with up to three attempts.
- Accuracy (paper section 7.2) is agreement with curated reference files, `syntax_specs/<command>.py`, built from manual pages (the paper: two graduate students, 80 person-hours; the repository: 91 commits by four authors, with messages such as "Fix syntax_spec ground truths").

This is a sound design for its purpose: the syntax is fixed, reviewable and reproducible, and anchored to the documented interface. The case studies ask what is left out when the documented interface and the binary's actual interface differ.

1.2 The runs

Seven runs of the agent can be re-read in full from their logs (two earlier runs' records no longer exist). All used the same first-version prompt, in the same base virtual-machine image, with permissions bypassed so the agent could act freely; runs 6 and 8 used commands installed into that image from Ubuntu's package archive. This section uses runs 3 to 5; the later sections use the rest, and each repeats its own runs' details.

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
3	`grep`	Claude Haiku 5.5	open	739 s · $0.96	context only: it found this project's public notes on `grep` through a GitHub search
4	`grep`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	322 s · $2.54	yes, section 1
5	`rm`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	60 s · $0.80	yes, section 1
6	`binwalk`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	193 s · $2.12	yes, section 2
7	`cut`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	104 s · $1.25	yes, section 4
8	`convert`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	1,291 s · $6.04	yes, section 3
9	`sort`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	304 s · $2.46	yes, section 4

In section 1's runs both commands were among the four examples, so the agent had a finished answer in front of it: for the syntax, that answer is Caruca's own reference file, byte for byte. The case studies therefore report where the agent *departed* from the reference, and why. None of the commands in runs 6 to 9 is an example.

1.3 Case study A: ask the command, not only its manual

In both runs the agent read the manual, ran `--help`, and then ran the binary on small inputs before writing anything: 72 runs for `grep`, 27 for `rm`. Its probes were aimed at the places a syntax description is hardest to get right: options with optional values, invalid values, negative numbers, abbreviated option names, grouped flags, `--`. In `rm`'s case it passed a deliberately invalid value (`--interactive=bogus`), and `rm` answered with the complete list of values it accepts.

Six facts about the two commands' argument syntax, and where each one is recorded:

FACT (GNU COREUTILS 9.4, GNU GREP 3.11)	MANUAL PAGE	CARUCA REFERENCE	BINARY	AGENT'S SYNTAX FILE
`rm --interactive` also accepts `no`, `none`, `yes`	no (lists `never`, `once`, `always`)	no (3 values)	yes, listed in its own error message	6 values
`rm` may be given no file at all (`rm -f` exits 0)	yes: `rm [OPTION]... [FILE]...`	no: at least one file	yes (without `-f` it refuses to run)	zero or more files
`grep -y` means `-i`	no	absent	yes	present
`grep -u` / `--unix-byte-offsets` exist	no	absent	yes, with an "obsolete" warning	present
`grep --color` takes its value only after `=`	yes: `--color[=WHEN]`	value as a separate word	yes: `--color never` searches for "never"	value after `=`
`grep -A`, `-B`, `-C` reject negative numbers (while `-m -1` is legal)	not stated	no lower bound on any of the four	yes	`min=0` on the three, none on `-m`

The manual pages are those bundled in Caruca's repository (`grep` 3.11, the same version as in the virtual machine; `rm` from coreutils 8.32, whose option list matches 9.4's). "Binary" is the agent's own probe, verified in its session log. Caruca's model output for both commands (`outputs/llm-dsl-generation/`) records none of the six either.

What the table shows.

- **Two of the six were in the manual all along** (`rm`'s optional file list and `--color[=WHEN]`). The reference files do not record them, and neither does the model's output from the same manual. Running the binary turned each into a checked fact rather than a reading of documentation notation.
- **Four are in the binary only** (`rm`'s extra values, `-y`, `-u`, the context options' refusal of negative numbers). For the first three, Caruca's syntax step cannot find them by design: its input is the manual, and its prompt forbids values the manual does not list. The agent found `rm`'s extra values in a single invalid call, by letting the command describe its own interface.
- **The yardstick question.** Scored against Caruca's references, the agent's departures count as errors; scored against the binary, they are corrections. The references measure faithfulness to the documentation; the tools that consume Caruca's output run against the binary. Both are legitimate targets, and the paper currently reports only the first.

The agent did not get everything right: in PaSh's parser table it kept the example's separate-word `--color WHEN` and `--interactive WHEN` forms although its own probes showed them wrong for this binary, and it said so in its report. The point of the case study is the method, not a score.

1.4 Case study B: `grep`'s `-e` / `-f` form

`grep`'s manual lists three forms in its synopsis (Caruca's bundled copy, `doc_sources/man/grep.txt`, lines 8–10):

```
grep [OPTION...] PATTERNS [FILE...]
grep [OPTION...] -e PATTERNS ... [FILE...]
grep [OPTION...] -f PATTERN_FILE ... [FILE...]
```

In the second and third forms there is no pattern operand: every operand is a file. Loaded through Caruca's own syntax module, each of its three `grep` grammars has exactly one form, a required pattern operand followed by zero or more files: the model's output (`outputs/llm-dsl-generation/grep.py`), the curated reference (`syntax_specs/grep.py`), and the subset used for PaSh (`pash_syntax_specs/grep.py`). The first two list `-e` and `-f` as options while still requiring the pattern operand; the third lists neither. Run at one flag at a time, Caruca's generator produces 122 `grep` command lines, and the one using `-e` is `grep -e a a`, in which `grep` reads the second `a` as a file name, not a pattern. Because the reference has the same single form as the model's output, the syntax comparison of paper section 7.2 reports a match: the gap is invisible to the evaluation.

What the agent did (run 4, no web). It wrote two syntaxes sharing one option list: the pattern as the first operand, and `-e` / `-f` with every operand a file. The log shows where this came from, and it was not the synopsis, which the agent never displayed (it printed only the manual's option lines and `--help`, whose usage line shows one form). Two sources in front of it carried the fact:

1. **PaSh's hand-written `grep` annotation**, given to the agent as part of the example, already handles it: if `-e` or `-f` is present, every operand is an input file.
2. **The binary**: its probe `grep -e beta t.txt t.txt` searched both operands as files.

So the agent did not discover the form from nothing. It noticed that two descriptions of the same command disagree (the grammar has one form, the behaviour annotation two), checked which one the binary agrees with, and made the grammar match. In Caruca's architecture the grammar and the annotations are produced in different stages and never compared with each other, so this kind of disagreement has no place to surface.

(Run 3, with web access, also wrote several forms, but only after reading this project's own notes on the gap through a GitHub search, so it is not used as evidence.)

1.5 What this suggests for Caruca

Each suggestion reuses something Caruca already has.

1. **Check the syntax against the binary.** Caruca already runs the real binary in an isolated sandbox during tracing. A short syntax check before tracing (each spelling once, each value form, each documented synopsis form, one invalid value per option) would catch documentation-notation misreads such as `--color[=WHEN]` and missing forms such as `grep -e`. In run 4 the agent's equivalent check was 154 runs of the binary, a few seconds.
2. **Let the command list its own values.** Where a command answers an invalid value with the list of valid ones, as GNU tools commonly do, that list is better evidence than the manual's. This needs the prompt's "no choices not in the manpage" rule relaxed to "no choices not confirmed by the manpage or the binary".
3. **Report syntax accuracy against two yardsticks.** Agreement with the reference (as now) and agreement with the binary, with the disagreements between them listed as a category of their own. That turns the 80 person-hours of reference annotation into a measurement of the documentation as well as of the model.
4. **Cross-check the grammar against the consumer's annotation where one exists.** PaSh's hand-written annotations encode facts about argument shape (as for `grep -e`) that a grammar can be checked against mechanically.

1.6 What this section does not claim

- **No rate.** This section rests on three runs (3 to 5), two commands, one prompt version; each combination of model and command ran once. These are case studies, not frequencies.
- **No claim the agent is more accurate than Caruca overall.** Neither run's output has been scored, the agent had the reference in front of it, and it made its own mistakes (end of section 1.3).
- **No claim the facts generalise beyond GNU tools.** Case study A relies on informative error messages; an undocumented binary may give none.
- **No claim about the rest of Caruca's pipeline.** The agent was weakest exactly where Caruca is strongest: it isolated nothing beyond a scratch folder, even for `rm`, and in neither run did it check the files it had written against the binary. That contrast is a candidate for a later section under this heading, once repeated runs show whether it is a pattern.

Evidence

CLAIM	SOURCE
Caruca's syntax input, prompt wording, retries	v1 `caruca/src/caruca/llm.py` (lines 28–50, 103–131, 155–159), commit `d80324073`
109 bundled `--help` files, unused by the code	v1 `caruca/src/caruca/doc_sources/helpfiles/`; no `.py` or `.sh` file references them
Reference files are curated	v1 `git log -- caruca/src/caruca/syntax_specs/`: 91 commits, 4 authors
The examples are Caruca's reference files	`naive_llm_caruca/samples/{grep,rm}/*.py`, byte-identical to v1 `syntax_specs/{grep,rm}.py`
`grep`'s three forms; three grammars with one	v1 `doc_sources/man/grep.txt` lines 8–10; the three grammar files, loaded through v1's `caruca.ir.syntax` (2026-10-20)
`grep -e a a` among 122 generated lines	`caruca generate grep --max-count 1`, v1 at `d80324073`, run 2026-10-20
Run 4's probes, files and sources	`naive_llm_caruca/runs/2026-10-20_112956_naive-run-grep-noweb_grep_PROMPT_v1/` (calls 2, 4, 5, 6, 7) and its `analysis/analysis.md`
Run 5's probes and files	`naive_llm_caruca/runs/2026-10-20_121932_naive-run-rm_rm_PROMPT_v1/` (calls 2, 3, 4) and its `analysis/analysis.md`
PaSh's `grep` annotation handles `-e` / `-f`	`naive_llm_caruca/samples/grep/annotation/InputOutputInfoGeneratorGrep.py` (from `pash-annotations` 0.2.4; the folder was `samples/grep/pash/` when the runs were made)
The agent's setup and prompt	`naive_llm_caruca/README.md`; `ai_docs/tasks/016_naive_claude_code_arm.md`

2. A command beyond the core utilities: `binwalk`

Section 1's commands are GNU core utilities, and in both runs the agent had a finished example for the command in front of it. Run 6 removes both conditions. `binwalk` scans binary files (firmware images, disk images, archives) for embedded files and executable code, and can extract what it finds. It is not one of the 120 commands Caruca has syntax specifications for (`syntax_specs/` holds 120 files), so there is no Caruca reference, model output or annotation for it, and none of the four examples resembles it. It was installed from Ubuntu's package archive, as a user would install it. The setup was otherwise that of runs 4 and 5: the same first-version prompt, model, virtual-machine image and no web access.

Summary

- **Whether a command is opaque depends on how it is packaged.** The installed `binwalk` is a four-line Python script that loads binwalk's Python package. The agent read that package and ran binwalk's own option-registration code to list every option with its type, and wrote its grammar from that list. Caruca's subject is opaque components; many commands installed from a package manager are not.
- **The documentation lags the program.** The manual lists 49 options and describes version 2.3.2; the installed program lists 51 and reports 2.3.3; the package is 2.3.4. As in case study A, a documentation-only syntax step cannot see the two missing options.
- **binwalk's behaviour lives in its input's contents.** It reports only what carries a known format signature. Run on the eight files Caruca fills its test folders with, binwalk found nothing in seven and only the image's own header in the eighth. The agent's first probe built a file worth scanning (a gzip stream followed by a tar archive).
- **Two grammar details matter to Caruca's later stages.** A correct integer range in the agent's grammar would make Caruca's generator try 65,536 values for one option; and binwalk can run arbitrary programs (`-D type:ext:cmd`), which Caruca's own grammars would mark with their `Command` type and the agent's grammar does not.
- **What it suggests:** treat file content as part of the environment model for commands that inspect formats; sample typed ranges instead of enumerating them; type command-valued arguments as `Command`; and state, per command, whether the evaluation treated it as opaque.

2.1 The run

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
6	`binwalk` (Ubuntu package 2.3.4+dfsg1-5)	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	193 s · $2.12	yes

The agent made 18 tool calls. It read the formats and examples, ran `--help`, read the manual, read binwalk's installed source (calls 6 to 9), ran the binary 30 times in a scratch folder, wrote all four files in one call, then tested them and made one correction after four more runs. It used no network.

2.2 Where the syntax came from

The prompt pointed the agent to the binary and the manual. Its fifth call showed the binary is a Python script (`from binwalk.__main__ import main`), and from then on the installed package was its main source. In call 9 it imported binwalk's module registry and printed each module's command-line option table: every short and long spelling, whether the option takes a value, and the value's Python type (`int`, `float`, `str`, or `list` for an option that may be repeated). Its syntax file and PaSh table follow that table one for one. It compared the table with the manual and with `--help`:

SOURCE	VERSION IT STATES	OPTIONS LISTED
Ubuntu package (`dpkg`)	2.3.4+dfsg1-5	—
`binwalk --help`	v2.3.3	51
Manual page (generated by `help2man` , September 2021)	2.3.2	49: lacks `-0` / `--run-as` and `-1` / `--preserve-symlinks`
binwalk's own option table (call 9)	—	51 (plus one hidden option, which crashes from the shell)

Two observations follow.

- **The source answered the syntax question better than the manual did.** For an interpreted command, the option parser's own table is the definitive list of spellings and value types, and it was on the machine. A documentation-based syntax step, given the bundled manual, would have missed the two options added after 2.3.2.
- **But the agent's check could not fail.** Its main test compared the written files with the same option table they had been written from. It ran 16 of the 51 options on the binary at least once; the 35 never run include the ones with the largest effects: `-D` (runs a command for each match), `-r` (deletes extracted files), `-M` (extracts recursively), `-0` (runs extraction tools as another user) and `-1` (keeps symbolic links that point outside the extraction folder).

2.3 What binwalk needs from the environment

Caruca's configuration generator pairs each invocation with a file-system state and fills files from a fixed content pool. The paper (section 4.3) describes the pool as text, math formulas, JSON and "a set of PNG images ranging from 2kb to 4MB"; the code at `d80324073` ships eight files (`ir/contents.py` , `data/`): two text passages, two arithmetic files, two time files, one JSON file and one JPEG image. The default content level, `simple` , uses the text only.

We ran the installed `binwalk` on each of the eight files, in the virtual machine after the session:

CONTENT FILE(S)	BINWALK RESULTS
`human_1.txt` , `human_2.txt` , `math_1.txt` , `math_2.txt` , `time_1.txt` , `time_2.txt` , `json.json`	none
`usenix.jpg`	1: the JPEG header at offset 0 (the file's own format)

So at the default level every invocation Caruca generated for `binwalk` would print an empty table, and extraction, entropy plotting and file comparison would have nothing to act on. The paper's coverage section states the assumption this meets: path normalization holds because "a command behavior depends not on a specific path name, but rather the identified location type (e.g., file, directory, empty)" (section 7.3). For `binwalk` the location type is the same in every case; what changes the behaviour is what is inside the file.

This cuts both ways. Given an input that triggers extraction, Caruca's tracing would record what the agent's output cannot express: `binwalk -e fw.bin` creates `_fw.bin.extracted/` in the current folder. The agent observed this, but PaSh's input/output interface (`pash-annotations` 0.2.4) has calls only for operands, option arguments and standard input and output, so its annotation records the folder in a comment alone. Traced reads and writes are where Caruca's design is stronger than the agent's.

2.4 Two grammar details with consequences for the pipeline

A typed range. The agent wrote binwalk's status-server port as `Integer(flag="-s", min=0, max=65535)`, which is correct. Caruca's `Integer` generates every value in its range (`ir/syntax.py`, lines 263–269; the default range is –1 to 1, which is what the paper's section 4.2 describes). Fed the agent's grammar, Caruca's own generator (`string_invocations`, at most one option per invocation) produces 65,659 command lines, of which 65,536 are `binwalk -s <port>`, each starting the status server on a different port. Without `-s` it produces 122, the same order as `grep`'s 121 by the same method. None of Caruca's 120 grammars uses `min` or `max`, so the question has not arisen with hand-written grammars; it arises as soon as a model writes a correct range.

An option that runs programs. `binwalk -D type:ext:cmd` runs `cmd` for every match: after the session we ran `binwalk -q -D "gzip:gz:touch RAN_%e" c.bin` and it created `RAN_p.txt.gz`. Caruca's grammar language already has a type for this: `Command` (`ir/syntax.py`, line 387), which its own grammars use for option values such as `sort --compress-program`, `split --filter`, `pandoc --filter` and `install --strip-program`. The agent typed `-D`'s value as free text, and none of its four files says that binwalk can run other programs. The paper names such commands (`xargs`, `nohup`) as the case where its string normalization does not hold (section 7.3).

2.5 What this suggests for Caruca

Each suggestion extends a mechanism Caruca already has.

1. **Make content part of the environment model for format-inspecting commands.** The content pool already varies text, maths, JSON and images. Adding a few format-bearing samples (a compressed stream, an archive, a small file-system image) would let a command like `binwalk` exercise its extraction paths, and Caruca's tracing would then record the files they create. The syntax step could mark a command as content-sensitive, much as it already types arguments, to select them.
2. **Sample typed ranges instead of enumerating them.** Generating the two endpoints and a value inside the range, in the spirit of the –1, 0, 1 the paper uses for integers, keeps `min` / `max` safe for model-written grammars without losing the bound.
3. **Type command-valued arguments as `Command`, and report them.** The type exists and Caruca's references use it. A grammar check that flags option values described as commands in the manual would catch cases like `-D`, and the evaluation could list such commands with `xargs` and `nohup` as outside string normalization.
4. **State the evidence condition per command.** Whether a command was treated as opaque (binary and documentation only) or its source was available changes what a syntax step can know. For interpreted commands the option parser's own table is a third yardstick for syntax, alongside the reference and the binary of section 1.5.

2.6 What this section does not claim

- **One run, one command.** binwalk ran once, with one prompt version. It has no Caruca reference, so nothing here is scored.
- **Some checks are ours, not the agent's.** The package version, `-D`'s command execution and the runs on Caruca's content pool were checked by us in the same virtual machine after the session; the session log does not contain them.
- **The 65,536 figure is Caruca's generator applied to the agent's grammar.** It says nothing about Caruca's own grammars, none of which uses a range.
- **No claim that most commands ship as source.** binwalk is one interpreted command; many packaged commands are compiled.
- **The agent repeated section 1's weaknesses.** It isolated nothing beyond one scratch folder (whose creation was not checked, and one probe wrote a cache file outside it), tested its files mainly against its own source, stated "Stdin is never read" although its own probe read `/dev/stdin` from a redirected file, and did not mention `-D`.

Evidence

CLAIM	SOURCE
binwalk is not among Caruca's grammars	v1 `caruca/src/caruca/syntax_specs/` : 120 grammar files, none for `binwalk`
The binary is a Python script; call 9's option table; 51 options; manual 2.3.2 with 49; `--help` v2.3.3	`naive_llm_caruca/runs/2026-10-20_133813_naive-run-binwalk_binwalk_PROMPT_v1/` (calls 5, 6–9)
Package 2.3.4+dfsg1-5	`dpkg -s binwalk` in the run's virtual machine, 2026-10-20, recorded in the run's `analysis/analysis.md`
30 runs before writing, 4 after; 16 of 51 options run; the circular test	the run's transcript (calls 10, 11, 16, 17) and `analysis/analysis.md` section 2.4
Caruca's content pool	v1 `caruca/src/caruca/ir/contents.py` lines 11–52 and `data/` , commit `d80324073` ; the paper, section 4.3
binwalk on the pool: nothing in seven files, the JPEG header in one	binwalk 2.3.3 in the run's virtual machine, 2026-10-20
`-D` runs a command	`binwalk -q -D "gzip:gz:touch RAN_%e" c.bin` in the run's virtual machine, 2026-10-20
`Integer` enumerates its range; `Command` type	v1 `caruca/src/caruca/ir/syntax.py` lines 263–269 and 387
`Command` used for option values in Caruca's grammars	v1 `syntax_specs/{sort,split,pandoc,install}.py`
Generator counts (65,659; 122	v1's `caruca.cli.generate.string_invocations` at `d80324073` , run on the agent's `output/binwalk.py` from a scratch package, 2026-10-20

CLAIM	SOURCE
without `-s`; `grep` 121)	
PaSh's input/output interface	`pash_annotations/annotation_generation/annotation_generators/InputOutputInfoGenerator_Interface.py`, `pash-annotations` 0.2.4
Path normalization and higher-order commands	the paper, section 7.3

3. A wide interface: `convert`

The paper singles out one command in its cost evaluation. With at most two options per invocation, "the sole outlier is `convert`, with a projected runtime of 98.72 hours", and the paper explains why this is not a concern: `convert` "has a simple specification, i.e., it just reads a single file which it converts and writes to a different output file, but has a very complex interface", so "the exhaustive flag combinations exaggerate its complexity without revealing meaningful behavioral variation" (section 7.4). Run 8 gave the agent this command. `convert` is one of Caruca's 120 commands, so a reference grammar and a model-written grammar exist to compare with; unlike section 1's commands, it is not one of the four examples, nor one of the four worked examples in Caruca's own syntax prompt (`touch`, `rm`, `mv`, `ls`), so neither the agent nor Caruca's model had its answer in front of it. The setup was otherwise that of runs 4 to 6.

Summary

- **The agent spent its effort where the paper says the complexity is: the interface.** It took the manual's option list, compared it with `convert -help`, tried 352 words from ImageMagick's library as option names, and kept the 261 the program accepts. It then measured how many words each option takes and took each option's allowed values from the program's own catalogues. About 3,700 runs of `convert`, one option at a time, before writing anything.
- **The manual and the program disagree, and Caruca's grammars follow the manual.** Caruca's reference and its model output both list exactly the manual's 248 options. One of those, `-connected-component`, is rejected by the program (it is `-connected-components`); `-view`, a flag in both grammars, takes a value; and 10 accepted options appear in neither the manual nor `-help`. This extends section 1's case study A from two GNU tools to a large non-GNU one.
- **Option order is part of `convert`'s syntax, and the grammar language can hold it.** `convert` accepts options before, between and after its files. Caruca's grammar language is a sequence of positions, and the grammar Caruca uses for its PaSh tests of `convert` has three: an input file, an optional `-resize`, an output file. The reference and model grammars, and the agent's, use two (options, then files). Fed through Caruca's generator, the layout decides whether a generated line converts anything at all.
- **PaSh's parser cannot hold option order.** It stops reading options at the first file name, so in the order `convert in.png -resize 50% out.jpg` the option words reach the annotation as file names.
- **What it suggests:** extend section 1's binary check with the program's own help text and library as candidate sources; use positions to describe commands whose options sit between files; for wide interfaces, test each option alone against the binary before combining any; and record, per command, which option orders a consumer's annotation is valid for.

3.1 The run

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
8	`convert` (ImageMagick 6.9.12-98, Ubuntu package `8:6.9.12.98+dfsg1-5.2build2`)	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	1,291 s · $6.04	yes

The agent made 33 tool calls. It read the formats, all four examples and the whole manual page, then spent calls 5 to 23 on the binary: comparing option lists, finding accepted names, counting each option's words, and checking standard input and output. A script of its own wrote the grammar and PaSh's parser table from those measurements; the two PaSh classes were written by hand. It then tested the files with a harness of its own, about 3,100 more runs. It used no network. (The session time is Claude Code's own measure; the run took longer on the clock because the host machine slept, which paused the virtual machine without affecting the session.)

3.2 Where the syntax came from

SOURCE	OPTIONS	NOTES
Manual page (`convert-im6.q16.1.gz`, 324 lines)	248	includes `-connected-component`, which the program rejects; lists `-view` with no value
`convert -help`	250	4 not in the manual (`-connected-components`, `-kuwahara`, `-local-contrast`, `-rotational-blur`); 2 manual names missing (`-connected-component`, `-metric`)
Words in ImageMagick's library (`libMagickWand`)	352 candidates	tried as names; 11 accepted that `-help` lacks, 10 of them also absent from the manual (`-average`, `-cache`, `-concurrent`, `-duration`, `-gaussian`, `-linewidth`, `-maximum`, `-minimum`, `-noop`, `-recolor`)
The program, by trial	261 accepted	each also in a `+` form; 11 `-` forms and 3 `+` forms take two words
Caruca's reference grammar (`syntax_specs/convert.py`)	248	the manual's list exactly; no `+` forms; `-view` as a flag; one operand node, one or more files
Caruca's model output (`outputs/llm-dsl-generation/convert.py`)	248	the same 248 names and the same flag-or-value split as the reference; two single-file operands
The agent's grammar	261 (522 with `+`)	`-view` takes a value; operands: input files, then one output file

We rechecked the disputed points on the same build after the session: `convert in.png -connected-component 4 out.png` fails with "unrecognized option", and `convert in.png -view out.png` treats `out.png` as `-view`'s value and writes nothing.

Two observations follow.

- **The program describes itself in more than one place.** For `rm` (section 1.3) the program's error message listed its values. For `convert`, `-help`, the library's strings and the `-list` catalogues (`convert -list Colorspace` and 38 others) together gave the option names and the allowed values. A documentation-only syntax step sees none of these. Caruca's model wrote the same name list as the reference, from the same manual.

- **The agent's measurement was thorough, its summary of it was not.** Its closing report says all 522 spellings passed its final test. Its harness skipped 13 of them, and its last pass still flagged one (a test with a wrong value, not a wrong grammar). As in section 1, the agent's own account of its checking is not evidence; the log is.

3.3 Option order and Caruca's generator

Caruca's grammar language describes a syntax as positions in order, each holding the nodes allowed there. The four example grammars, and Caruca's reference and model grammars for `convert`, use two positions: options, then operands. Caruca's PaSh test grammar for `convert` (`pash_syntax_specs/convert.py`, unchanged since 2024-11-28) uses three:

```python
convert_syntax_spec = [[
    [Path(arity=Arity.EXACTLY_ONE)],     # input file
    [Geometry(flag="-resize")],          # options between the files
    [Path(arity=Arity.EXACTLY_ONE)],     # output file
]]
```

So the language already expresses the order `convert in.png -resize 50% out.jpg`, and Caruca's authors used it for this command.

The layout matters downstream. We fed each of the three full grammars to Caruca's generator (`caruca generate convert --max-count 1`, default settings, on the Mac; the agent's and the model's grammars loaded from a scratch package with `--path`):

GRAMMAR	GENERATED LINES (DISTINCT)	DISTINCT LINES WITH BOTH AN INPUT AND AN OUTPUT FILE
Caruca's reference (one operand node, one or more files)	7,308 (1,052)	0
Caruca's model output (two single-file operands)	14,616 (2,272)	2,272
The agent's (input files, zero or more, then one output)	63,048 (2,422)	4, all without an option

At the generator's default of at most one file per repeatable operand, the reference gives every line one file. With the agent's grammar, every line that carries an option has only the output file. We ran representative lines on the same build, with Caruca's own test image (`data/usenix.jpg`): `convert -colors 2 f.jpg` and `convert -resize 50% f.jpg` fail with "no images defined" and write nothing; `convert f.jpg` prints the version; `convert in.jpg out.jpg` converts. Section 7.4 does not say which grammar and generator settings its projection for `convert` used; at these defaults, the reference grammar's option lines would exercise no conversion. Caruca's PaSh tracing (`caruca/run.sh`, `caruca trace --pash`) used the three-position grammar instead, with the full content pool (`--content split`, which includes the one JPEG). Caruca's first committed PaSh annotation for `convert` (`caruca/save/convert.json`, 2024-11-28) reads "Found no successful runs. Defaulting to side-effectful"; the file has held no cases since 2024-12-04, and `benchmarks/MILESTONES` lists `convert` as not done. We did not establish why those runs failed.

3.4 What PaSh receives

PaSh 0.2.4 ships no annotation for `convert`; Caruca's evaluation files hold a hand-written one (`eval/pash-annotations/convert.json`): pure, reading the first argument and writing the second. The agent wrote PaSh's current three files. We loaded them into a copy of PaSh's own `pash_annotations` 0.2.4 and ran them on eleven invocations through PaSh's parser. They load and run without error, and they are right for options placed before the files, for `-` as standard input, for `png:-` as standard output and for `-version`. PaSh's parser, however, reads options only until the first word that is not one, so:

INVOCATION	WHAT PASH'S PARSER PASSES ON	WHAT THE AGENT'S ANNOTATION THEN RECORDS
`convert -resize 50% in.png out.jpg`	option `-resize 50%`; files `in.png`, `out.jpg`	correct
`convert in.png -resize 50% out.jpg`	files `in.png`, `-resize`, `50%`, `out.jpg`	`-resize` and `50%` as input files
`convert in.png -write side.png out.png`	files `in.png`, `-write`, `side.png`, `out.png`	`side.png` as an input, though `convert` writes it
`convert -set comment hi in.png out.png`	option `-set comment`; files `hi`, `in.png`, `out.png`	`hi` as an input file (the table allows one word per option)

The agent anticipated the first and last cases and described them in comments as accepted over-approximations. PaSh's own benchmark that uses `convert` (`eval/pash-scripts/dependency_untangling/img_convert.sh`) writes `convert -resize 70% "$i" "$out"`, options first, which the agent's annotation handles. The order in the PaSh test grammar of section 3.3 is the other one.

3.5 What this suggests for Caruca

Each suggestion extends a mechanism Caruca already has.

1. **Widen the binary check of section 1.5 to the program's other self-descriptions.** For each candidate name from the manual, `--help` and, where the program allows it, its listing commands, run the program once and keep what it accepts. For `convert` this corrects one name, adds ten and fixes one flag-or-value decision, at a few runs of under a second each per option.
2. **Use positions where options sit between files.** The grammar language and Caruca's own PaSh test grammar already do this. A grammar check could flag commands whose manual synopsis shows options after a file (`convert`'s reads `convert [input-option] input-file [output-option] output-file`), and the generator would then produce lines that convert something.
3. **For wide interfaces, test options one at a time before combining them.** This is the paper's own observation turned into a schedule: single-option runs establish the syntax cheaply (the agent's whole session, writing and testing included, took about 21 minutes of working time for 261 options in both forms), and combinations can be kept for the options whose single runs change which files are read or written.
4. **Record which option orders an annotation holds for.** Where a consumer's parser stops at the first file, as PaSh's does, an annotation for `convert` is valid for options-first invocations only. Saying so in the output lets the consumer decline the others instead of treating option words as files.

3.6 What this section does not claim

- **One run, one command, not scored.** The agent's grammar has not been scored against the reference or the binary beyond the points rechecked above.
- **Some checks are ours, not the agent's.** The package version, the rechecks of `-view` and `-connected-component`, the generator counts, the test-image runs and the PaSh runs were made by us after the session.
- **The generator figures are for one setting.** They use the generator's defaults at one option per line; a higher repeat limit gives the reference grammar lines with more files. Beyond four representative lines, the generated lines were not run.
- **The binary-run counts are estimates** from the logged outputs of the agent's loops, not a replay.
- **The agent repeated section 1's weaknesses.** It isolated nothing (a fixed folder in `/tmp`), ran every word from the library as an option, one of which started something that hung until stopped, and overstated its own testing.

Evidence

CLAIM	SOURCE
`convert`'s projected 98.72 hours and the paper's explanation	the paper, section 7.4
`convert` is not among the worked examples of Caruca's syntax prompt	v1 `caruca/src/caruca/llm.py` line 116, commit `d80324073`
The run: calls, sources, probes, report	`naive_llm_caruca/runs/2026-10-20_135842_naive-run-convert_convert_PROMPT_v1/` (calls 3–23 for the sources, 24–31 for writing and testing) and its `analysis/analysis.md`
Package version	`dpkg-query` in the run's virtual machine, recorded in the run folder's `package_version.txt`
Manual 248 vs `-help` 250; library candidates; 261 accepted	the run's transcript, calls 5–7 and 13
`-connected-component` rejected; `-view` takes a value	rechecked on ImageMagick 6.9.12-98 in the run's virtual machine, 2026-10-20
Reference and model grammars: 248 names, no `+` forms, `-view` a flag	v1 `syntax_specs/convert.py` and `outputs/llm-dsl-generation/convert.py`, loaded through v1's `caruca.ir.syntax`, 2026-10-20
The three-position PaSh test grammar	v1 `caruca/src/caruca/pash_syntax_specs/convert.py` (commit `38aefafd0`, 2024-11-28, unchanged since); traced by `caruca/run.sh`
Generator counts	`caruca generate convert --max-count 1` (and `--path` for the agent's and model grammars), v1 at `d80324073`, on the Mac, 2026-10-20; repeat limit default 1 in v1 `caruca/src/caruca/cli/__init__.py` lines 27–30
Single-file lines fail, two-file lines convert	ImageMagick 6.9.12-98 in a fresh clone of the run's image, with v1's `caruca/src/caruca/data/usenix.jpg`, 2026-10-20
Caruca's committed `convert` annotation history	v1 `git log -- caruca/save/convert.json`: `38aefafd0` (2024-11-28, "Found no successful runs"), `1c22d9721` (2024-12-04, no cases), `6ceda7586` (2024-12-08, no cases); `benchmarks/MILESTONES`
Hand-written PaSh annotation for `convert`	v1 `eval/pash-annotations/convert.json`
PaSh 0.2.4 has no `convert` annotation; its parser stops at the first operand	`pash_annotations/annotation_generation/AnnotationGeneration.py` (command list) and `parser/parser.py` (`parse`), `pash-annotations` 0.2.4 in the pash VM
The agent's classes in PaSh's code, eleven invocations	a copy of `pash-annotations` 0.2.4 in the pash VM with the agent's three files added, 2026-10-20; recorded in the run's `analysis/analysis.md` section 2.4
PaSh's benchmark uses options first	v1 `eval/pash-scripts/dependency_untangling/img_convert.sh` line 9

4. Two commands with no example to copy: `cut` and `sort`

Runs 7 and 9 return to GNU core utilities, but to two that were not among the agent's four examples, so it had no finished answer in front of it. Both commands have everything Caruca holds for a command: a curated reference grammar, its model's output, and a committed PaSh annotation. Both are also commands PaSh annotates by hand in its current Python form (`pash-annotations` 0.2.4). That makes them the first runs where the agent's output can be set against all three. The setup was that of runs 4 to 6: the first-version prompt, the same model, virtual-machine image and no web access. (Run 8, `convert`, is the subject of section 3.)

Summary

- **Unprompted, the agent rebuilt Caruca's parallelizability test, and for `sort` it went further.** For `cut` it split a file into pieces, ran `cut` on each, joined the outputs and compared them with one run. That is the paper's definition of "stateless" (section 6.3). For `sort` it wrote a merge step (`sort -m` with the invocation's ordering options) and checked it on 20 option sets. The paper places that step, synthesising an aggregator, outside Caruca's scope (section 6.1).
- **Its PaSh files closely match PaSh's own hand-written ones, which it never saw.** The parser tables agree option for option, including how each file argument is used. The input/output classes agree in logic.
- **The consumer's own code found what the agent's tests could not.** Loaded into PaSh's real annotation code, the `cut` files give PaSh's answers. The `sort` merge gives no answer for any ordinary `sort`, because PaSh rejects the way it pairs its splitter and aggregator. PaSh also hides the error. The agent had tested only against stand-ins it wrote itself.
- **A syntax file is also input to Caruca's generator.** The agent's `cut` grammar records a rule Caruca's reference does not ("use one, and only one of -b, -c or -f"). But it marks short options as taking their value after `=`, a convention it took from the `uniq` example, which is Caruca's own reference file. Caruca's generator then writes command lines such as `cut -b=1`, and `cut` rejects them.
- **What it suggests:** run generated command lines once against the binary before tracing; generate the `=` form only for long spellings; check adapter output in the consumer's own code; consider a model-written aggregator, checked by the split runs Caruca already makes; and regenerate committed annotations after a change to the classification code.

4.1 The runs

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
7	`cut` (GNU coreutils 9.4)	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	104 s · $1.25	yes
9	`sort` (GNU coreutils 9.4)	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	304 s · $2.46	yes

For `cut` the agent read the manual and `--help`, ran `cut` 21 times on small files, wrote all four files in one call, and then ran its split-and-join test (45 runs). For `sort` it ran the binary 88 times before writing and about 960 times afterwards, in two test scripts of its own. Neither session used the network.

4.2 The agent's PaSh files against PaSh's own

PaSh ships hand-written annotations for both commands. The agent was given PaSh's files for its four examples only.

FILE	CUT	SORT
Parser table	identical, except that PaSh's lists a `-M` flag, which GNU `cut` 9.4 rejects ("invalid option")	identical, including each file argument's access kind (`--random-source` read as configuration, `--files0-from` naming the inputs, `-o` the output, `-T` a scratch folder), plus two entries PaSh's lacks: `-y` (accepted silently) and `--check`
Input/output class	the same three calls	the same logic, plus no standard output in check mode
Parallelizability class	the same two ways of splitting, except under `-z` (records end in NUL), where the agent declines to split	different: see 4.3 below

PaSh's annotation files are public, so the closeness may partly reflect what the model saw in training; the logs cannot separate that from derivation. The argument names in both tables are the manual's, and both orderings follow the manual's.

4.3 Parallelizability: the split test, and a merge

The paper defines the test (section 6.3): if a command has no side effects "and if the output of invoking C_i is identical to concatenating the outputs of invoking C_{i1}, …, $C_{i\,n}$, the command is stateless. Otherwise, it is pure." Caruca implements it over traces of an input and its two halves (`tracer/data.py`, lines 135–146 and 188–218). The paper also says where Caruca stops: telling a parallelisable pure command from a non-parallelisable one "amounts to being able to synthesize an aggregator, which is out of scope for Caruca but covered in prior work [15–17]" (section 6.1).

- **`cut`.** Without being asked, the agent ran the same test: a 5,000-line file split into four consecutive pieces, `cut` run on each, outputs joined, compared with one run, for five option sets. All matched. Caruca's committed annotation for `cut` (`caruca/save/cut.json`) agrees: all 35 of its cases are `stateless`.
- **`sort`.** Joining sorted pieces does not give a sorted whole, so `sort` is pure in the paper's sense. It is parallelisable with a merge. The agent wrote one: `sort -m` followed by the invocation's ordering options. It checked it by splitting the input into 2, 3 and 5 parts, sorting each, merging and comparing with one sort, on 20 option sets including keys, separators, numeric and month order. All matched. It also showed that with `-s` (stable) or `-u` (unique), interleaved pieces give a different result, and restricted those options to consecutive pieces. PaSh's own hand-written annotation uses the same merge.
- **Caruca's committed `sort` annotation** (`caruca/save/sort.json`) labels all six of its cases `stateless`. That file was committed on 2024-12-08. The check that requires each half's output to keep its input's line order, which a sort of unsorted input fails, was added to `tracer/data.py` on 2025-12-21 (commit `c97306bb4`, "fix parallelizability class classification"). The committed files may therefore not reflect the current classifier. We did not re-run Caruca's tracing to confirm.

4.4 The consumer's own code as the check

We loaded the agent's files into a copy of PaSh's annotation package (version 0.2.4, the installed copy left unchanged) and ran PaSh's own parser and lookup functions on the same invocations as with PaSh's files: 5 for `cut`, 12 for `sort`.

- **`cut`:** the same answers as PaSh's files in four of five; in the fifth (`cut -z -f 1`) the agent's class offers no split, by design.
- **`sort`:** for every ordinary invocation (`sort a.txt`, `sort -n -k 2 a b`, `sort -t : -k 2,2n -r`, and three with attached values) PaSh obtained **no parallelizability information at all**. PaSh's `Parallelizer` asserts that dealing lines out in turn (round robin) is paired only with plain concatenation or one of three adjacent-line aggregators, and

the agent paired it with its merge. PaSh's lookup catches every exception from an annotation class and returns nothing, so the failure is silent. Only `-s` and `-u`, where the agent had dropped round robin, gave an answer.

The agent could not have seen this. It had no copy of PaSh's package and tested against stand-ins it wrote itself, which carry no such rule. The format description we gave it lists the splitters and aggregators but not the rule, which is a gap in our materials. PaSh's own `grep` annotation, one of the examples, builds the same pairing, in a branch it never reaches. The agent did flag one PaSh issue correctly: listing `--check` as a valued option makes PaSh's parser take the next word as its value (confirmed: `sort --check a b` parses with the single file `b`).

The paper's downstream check for PaSh compared benchmark outputs by hash (section 7.1). A silent "no information" leaves the outputs unchanged, and only parallelism is lost, so a hash comparison cannot see this kind of failure.

4.5 The syntax file as input to Caruca's generator

A grammar is read twice in Caruca: by the matcher that checks real command lines, and by the generator that writes the command lines to trace. Two details decide what the generator writes:

- It writes each option with its first spelling only (`ir/syntax.py`, lines 117–121); the other spellings (`alias`) are used only in matching (lines 159–170, 340–355).
- `flag_followed_by_equals=True` makes it join option and value with `=` (`ir/mixin.py`, line 24), whatever that first spelling is.

So an option written `Range(flag="-b", alias=["--bytes"], flag_followed_by_equals=True)` is generated as `-b=1`, which GNU `cut` reads as the list `=1` and rejects. Caruca's own references use this form: `syntax_specs/uniq.py` on `-f`, `-s` and `-w` (and `uniq -f=1` is rejected: "invalid number of fields to skip"), and `syntax_specs/sort.py` on `-S`. The agent copied it from the `uniq` example, which is that reference file byte for byte.

We fed three `cut` grammars to Caruca's generator at its shipped limit of four options per command line, ran every distinct line once on GNU `cut` 9.4 with a three-line input, and counted the lines it accepted. Lines a shell could not re-read from their printed form (a quote character as delimiter) and lines naming Caruca's path placeholders were left out. "Doing real work" excludes lines that `--help` or `--version` short-circuits.

`CUT` GRAMMAR	DISTINCT LINES	JUDGED	ACCEPTED	DOING REAL WORK	MAIN REASONS FOR REJECTION
Caruca's reference	11,651	9,856	1,783	360	two selectors (6,450); `-w`, which the file marks "Only FreeBSD" (992); `-d` or `-s` without `-f` (490); no selector (141)
Caruca's model output	15,042	15,042	1,104	0	two selectors (6,500); delimiter written `-d=a` (6,485); `=` after `-b`, `-c` or `-f` (660)
The agent's (run 7)	13,222	12,555	2,160	0	delimiter written `-d=a` (9,150); `=` after `-b`, `-c` or `-f` (875); `-s` without `-f` (370)

The agent's three alternative syntaxes, one per selector, remove every "two selectors" and "no selector" line, which are 6,591 of the reference's rejections. Its `=` markers then reject the rest. It also missed two rules of the same kind as the one it found: `cut` accepts `-d` and `-s` only with `-f`. For `sort`, which needs no option at all, all three grammars yield mostly valid lines. At two options per line, the accepted share of judged lines is 70% for the reference, 85% for the model output and 53% for the agent's grammar, which again loses most to `=` after short options (`-t=:`, `-S=1k`).

These figures describe grammars as generator input. They are not a measure of how well a grammar describes the command: the agent's `cut` grammar states the selector rule correctly and still yields nothing usable.

4.6 What this suggests for Caruca

Each suggestion extends a mechanism Caruca already has.

1. **Run each generated command line once before tracing it.** Caruca already executes every configuration; one plain run on a fixed input, before the traced runs, separates command lines the binary rejects from those it accepts. Here that took one pass over 11,651 lines. The rejections are also feedback for the grammar: in `cut`'s case they name the missing rule ("only one list may be specified", "an input delimiter may be specified only when operating on fields").
2. **Generate `=` only where the spelling takes it.** Writing a `=`-marked option with its long alias (`--bytes=1`), or with a space after a short spelling, keeps the marker meaningful for options like `uniq --skip-fields` without producing `-f=1`.
3. **Use alternative syntaxes for "exactly one of" rules.** Caruca's grammar language already allows several syntaxes per command; the agent's three-syntax `cut` grammar shows the pattern.
4. **Check adapter output in the consumer's own code.** Loading each generated PaSh annotation through PaSh's lookup functions, and treating "no information" as a failure, catches what a benchmark hash comparison cannot.
5. **Consider a model-written aggregator, checked by tracing.** Caruca already runs commands on split inputs. A model proposing a merge, and Caruca checking it on those same splits, would let Caruca separate parallelisable from non-parallelisable pure commands, the distinction the paper currently leaves to prior work.
6. **Regenerate committed annotations after classification changes.** The committed `sort` and `cut` files predate the December 2025 classification fix.

4.7 What this section does not claim

- **One run per command, one prompt version.** Neither run's syntax file has been scored against Caruca's reference.
- **No claim that the agent derived PaSh's annotations.** They are public and may be in the model's training data.
- **The acceptance figures are narrow.** One three-line input per command line; exit status 0 means the binary accepted the line, not that its behaviour was traced or correct. They say nothing about Caruca's results in the paper, which used other grammars for PaSh (`pash_syntax_specs/`, traced by `run.sh`).
- **Caruca's current verdict on `sort` is not established.** We read the code and its history but did not re-run tracing.
- **The PaSh check covers 17 invocations** in a copy of version 0.2.4.
- **The agent's reports overstated again.** Its `sort` spelling test ended "BAD = 8" while the report says every form "worked". Its `cut` report says every option was run in every form (4 of 17 spellings were never run). It called an empty `--output-delimiter`'s output "a space … a quirk"; it is a NUL byte, GNU `cut`'s behaviour for an empty delimiter.

Evidence

CLAIM	SOURCE
Neither command among the examples; both have Caruca reference, model output and committed annotation	`naive_llm_caruca/samples/` (grep, rm, tee, uniq); v1 `syntax_specs/{cut,sort}.py`, `outputs/llm-dsl-generation/{cut,sort}.py`, `caruca/save/{cut,sort}.json`, commit `d80324073`
Run 7's probes, files, split-and-join test	`naive_llm_caruca/runs/2026-10-20_135253_naive-run-cut_cut_PROMPT_v1/` (calls 12–17) and its `analysis/analysis.md`
Run 9's probes, files, tests, `BAD = 8`	`naive_llm_caruca/runs/2026-10-20_160340_naive-run-sort_sort_PROMPT_v1/` (calls 15–22) and its `analysis/analysis.md`
PaSh's own `cut` and `sort` annotations	`pash_annotations` 0.2.4 in the `pash` virtual machine: `parser/command_flag_option_info/data/{cut,sort}.json`, `annotation_generation/annotation_generators/*{Cut,Sort}.py`
`cut -M` rejected; `cut -b1 -d:` rejected; empty output delimiter is NUL; `uniq -f=1` rejected	GNU coreutils 9.4 (Ubuntu 9.4-3ubuntu6.3) in the `pash` virtual machine, 2026-10-20
The split test and its line-order condition	the paper, section 6.3; v1 `caruca/src/caruca/tracer/data.py` lines 84–146, 188–218
Aggregators out of scope	the paper, section 6.1
Line-order check added 2025-12-21; committed annotations 2024-12-07/08	v1 `git log -S __preserves_line_order -- caruca/src/caruca/tracer/data.py`; `git log -- caruca/save/{cut,sort}.json`
PaSh results for the agent's files; round-robin assertion; silent lookup	copy of `pash_annotations` 0.2.4 with the agent's files, 2026-10-20; `annotation_generation/datatypes/parallelizability/Parallelizer.py` (`__init__`) and `annotation_generation/AnnotationGeneration.py`
First spelling only; `=` rendering; aliases in matching	v1 `caruca/src/caruca/ir/syntax.py` lines 117–121, 159–170, 340–355; `ir/mixin.py` line 24
`=` on short options in Caruca's references	v1 `syntax_specs/uniq.py` lines 10, 13, 16; `syntax_specs/sort.py` line 33; `naive_llm_caruca/samples/uniq/uniq.py` byte-identical to the former
Generator counts and acceptance	`PYTHONPATH=<scratch> caruca generate {cut,sort} --path <pkg> --max-count {1,2,4}`, v1 at `d80324073`, every distinct line run once on GNU coreutils 9.4 in the `pash` virtual machine, 2026-10-20
Benchmark outputs compared by hash	the paper, section 7.1